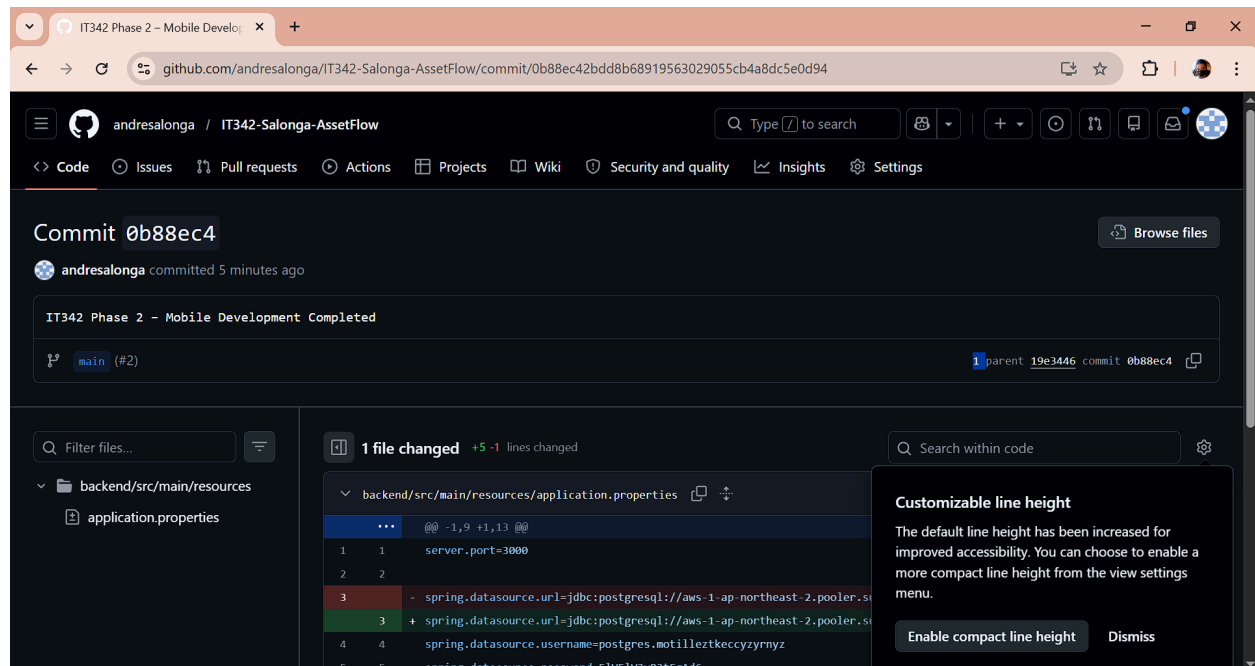


1. **GitHub Repository Link:** <https://github.com/andresalonga/IT342-Salonga-AssetFlow.git>

2. Final Commit for Phase 2

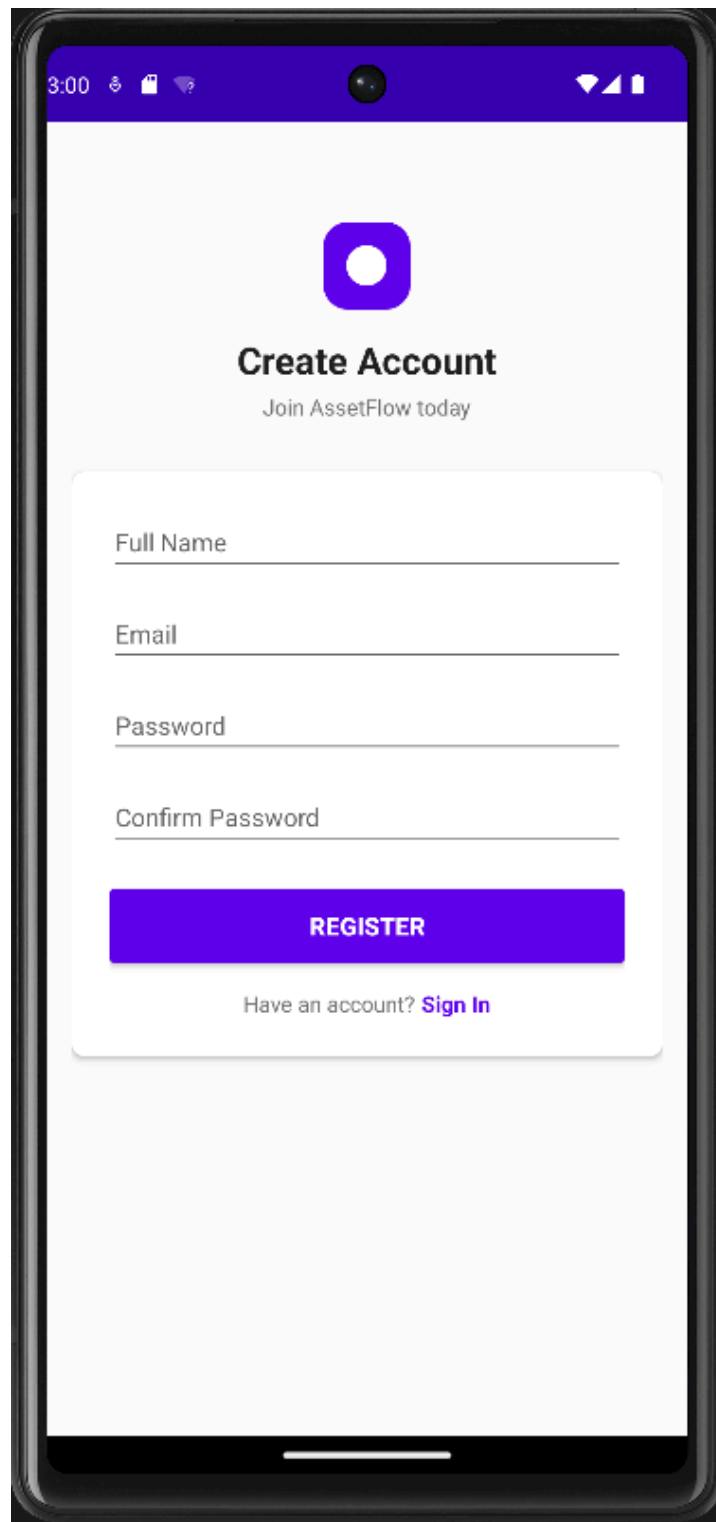
IT342 Phase 2 – Mobile Development Completed:

<https://github.com/andresalonga/IT342-Salonga-AssetFlow/commit/0b88ec42bdd8b68919563029055cb4a8dc5e0d94>



3. Screenshots

Registration screen



The screenshot shows a mobile application interface for creating a new account. At the top, there is a purple status bar with the time 3:00 and various system icons. Below this, a white header area contains a purple square logo with a white circle inside. The main title "Create Account" is displayed in bold black text, followed by the subtitle "Join AssetFlow today" in a smaller font. The registration form is contained within a white rounded rectangle and includes four input fields: "Full Name", "Email", "Password", and "Confirm Password". Each field has a thin purple underline. Below the input fields is a prominent purple button with the text "REGISTER" in white. At the bottom of the form, there is a link that says "Have an account? Sign In", where "Sign In" is in purple. The entire screen is framed by a black border representing the phone's bezel.

3:00



Create Account

Join AssetFlow today

Full Name

Email

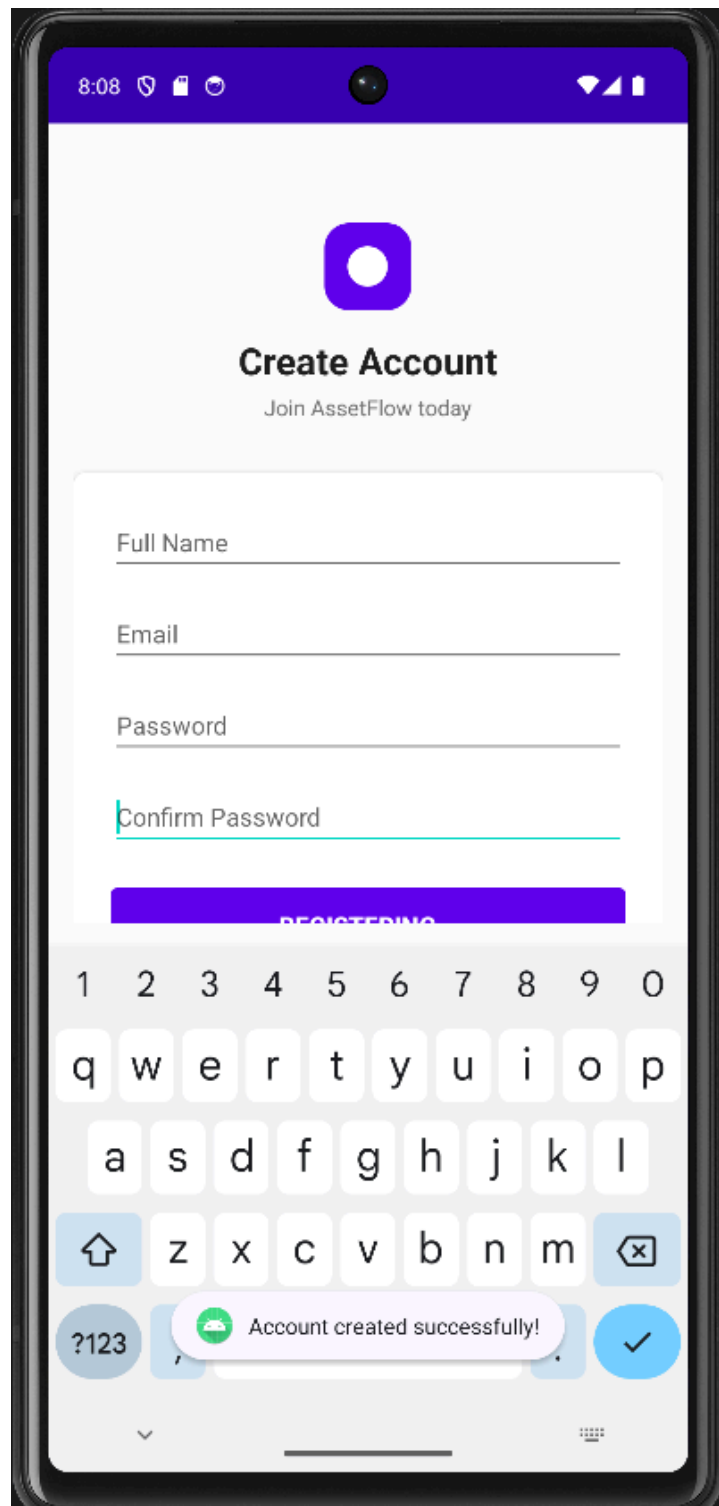
Password

Confirm Password

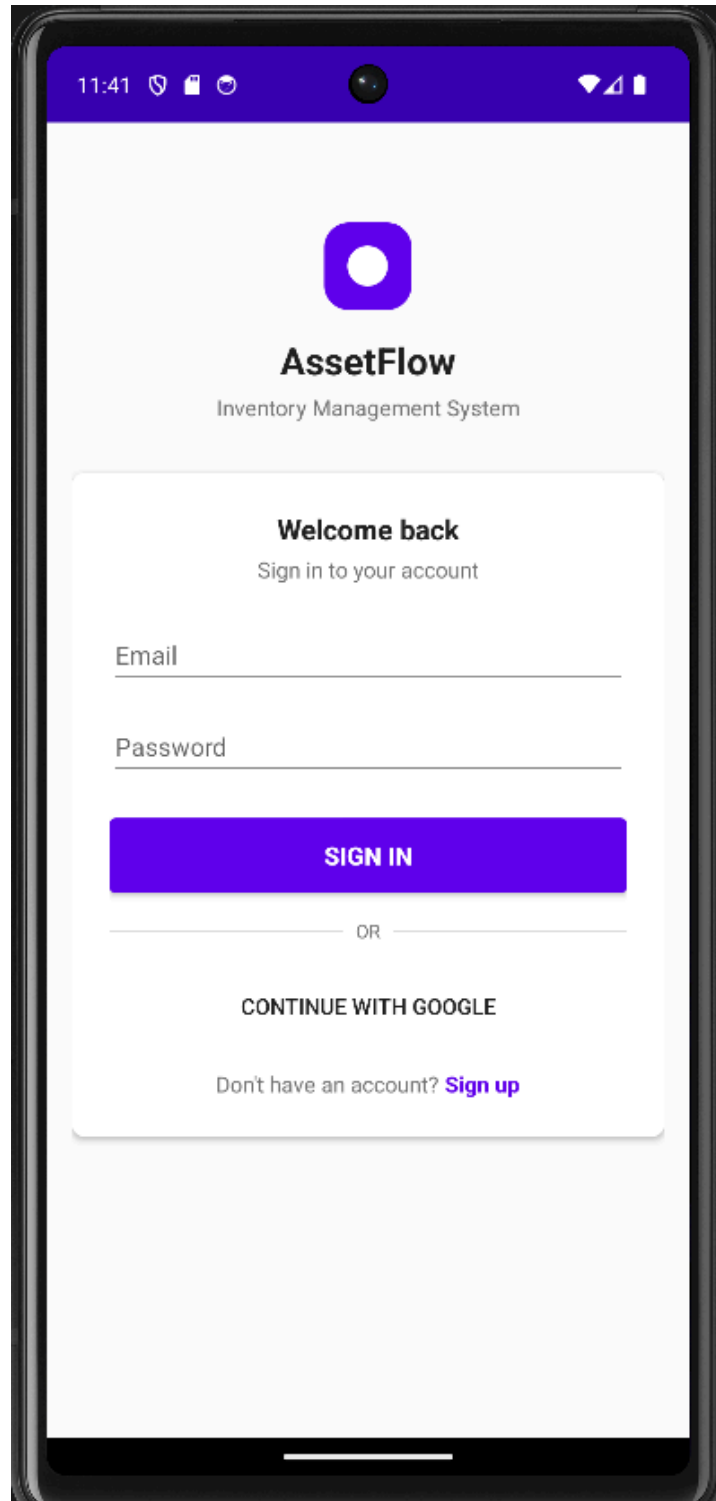
REGISTER

Have an account? [Sign In](#)

Successful registration



Login screen



The image shows a mobile application login screen for 'AssetFlow'. At the top, there is a purple status bar with the time '11:41' and various icons. Below this is a white header area with the 'AssetFlow' logo (a purple square with a white circle) and the text 'AssetFlow' and 'Inventory Management System'. The main content area is a white card with a purple border. It contains a 'Welcome back' message, a 'Sign in to your account' prompt, and two input fields for 'Email' and 'Password'. A prominent purple 'SIGN IN' button is located below the password field. Below the button is a horizontal line with 'OR' in the center. Underneath is the text 'CONTINUE WITH GOOGLE'. At the bottom of the card, it says 'Don't have an account? Sign up' with 'Sign up' as a link. The entire screen is framed by a black border representing the phone's bezel.

11:41

AssetFlow
Inventory Management System

Welcome back
Sign in to your account

Email

Password

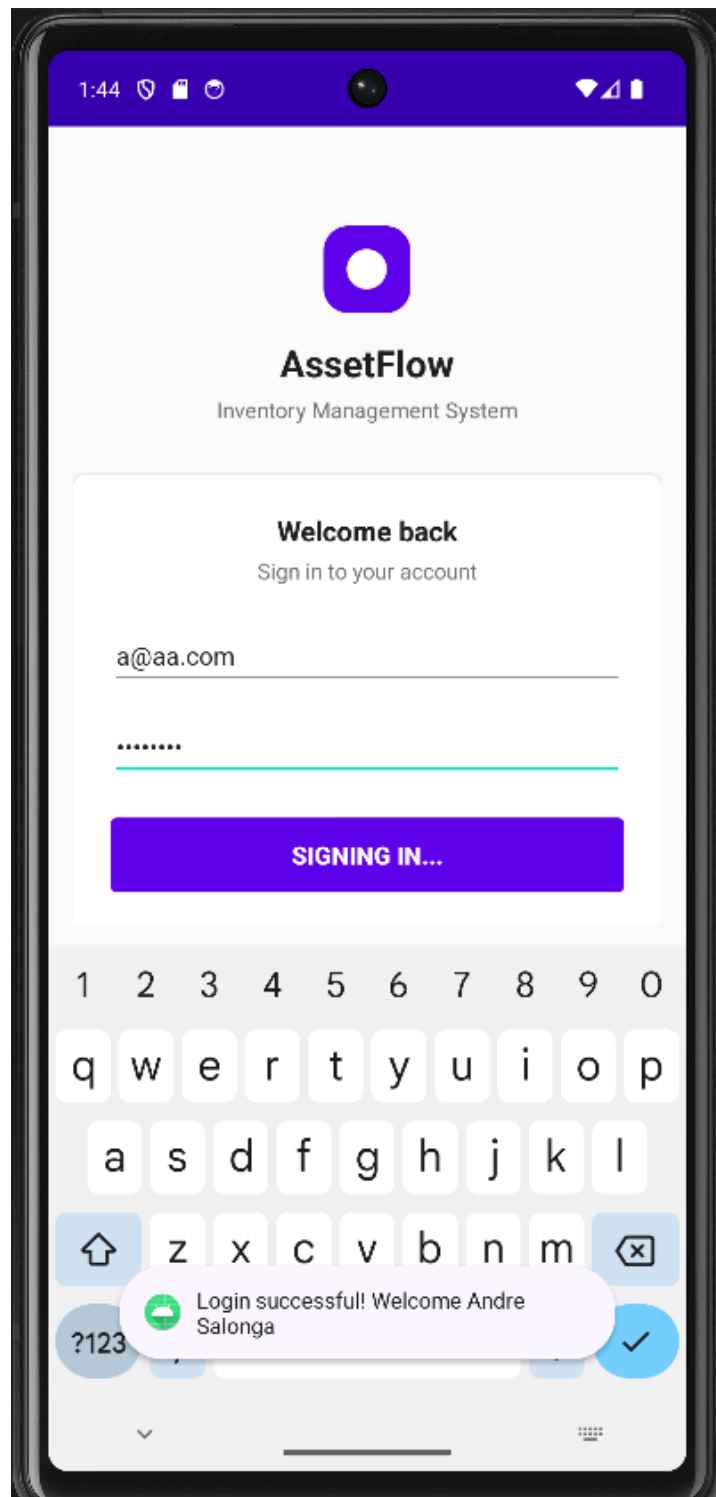
SIGN IN

OR

CONTINUE WITH GOOGLE

Don't have an account? [Sign up](#)

Successful login



After login screen



Database record

	id	name	email	password	created_at
☐	1	Andre Salonga	a@a.com	\$2a\$10\$L20YWx.YN7G5W00kbLzRGOC.2a	
☐	2	Admin Admin	admin@cit.edu	\$2a\$10\$eUIGUQGjxTKX8u7s6JDLFCsjUI.2a	
☐	3	Test Account	test@test.com	\$2a\$10\$cllQ1mFTAPO8Cuuen4se.wjc3G.2a	
☐	4	Register Test	register@test.com	\$2a\$10\$kILFfkY3dUqYBn9YOBARGO.hbrG.2a	
☐	5	My Test	my@test.com	\$2a\$10\$hKiEkbV3dalGYdkCviEKuuMDTF.2a	
☒	6	Andre Salonga	a@a.com	\$2a\$10\$h/qsaoy32bgT5Hz.XL0S.Crw/Y.2a	

4. Short Summary

How registration works

1. User Opens RegisterActivity
 - User enters: Full Name, Email, Password, Confirm Password
 - Input validation checks for empty fields and password match
2. Registration Request
 - RegisterActivity calls ApiClient.register(RegisterRequest) via Retrofit
 - Request sent to backend: POST /auth/register
 - Backend validates email uniqueness and password strength
3. Success Flow
 - Backend returns JWT token and user data
 - Token stored securely using TokenManager.saveToken()
 - User automatically navigated to DashboardActivity
 - Toast notification confirms successful registration
4. Error Handling
 - Invalid input → Toast error message
 - Network failure → Toast error message
 - Backend validation errors → Toast error message

How login works

1. User Opens LoginActivity
 - User enters: Email and Password
 - Can also navigate to RegisterActivity via "Sign Up" link
2. Login Request
 - LoginActivity validates input (not empty)
 - Calls ApiClient.login(LoginRequest) via Retrofit
 - Request sent to backend: POST /auth/login
3. Success Flow
 - Backend authenticates credentials and returns JWT token
 - Token stored via TokenManager.saveToken(token, email)
 - User redirected to DashboardActivity
 - Toast confirms successful login
4. Token Persistence
 - SplashActivity checks if token exists on app startup
 - If valid token → Navigate to Dashboard (already logged in)
 - If no token → Navigate to Login screen

API integration used

Base Configuration

- API Endpoint: http://10.0.2.2:3000/ (Android emulator access to localhost)
- HTTP Client: Retrofit 2.10.0 + OkHttp 4.11.0
- JSON Serialization: Gson 2.10.1

Network Security

- ClearText Traffic: Enabled for emulator testing via network_security_config.xml
- Request Logging: HttpLoggingInterceptor logs all API requests/responses

API Endpoints

- POST /auth/register — User registration with email/password
- POST /auth/login — User login with credentials
- GET /auth/me — Verify token and get current user

Request/Response Models

```
data class RegisterRequest(  
    val fullName: String,  
    val email: String,  
    val password: String  
)
```

```
data class LoginRequest(  
    val email: String,  
    val password: String  
)
```

```
data class AuthResponse(  
    val token: String,  
    val email: String,  
    val fullName: String  
)
```

Token Management

- Tokens stored in SharedPreferences via TokenManager singleton
- Token cleared on logout
- Token included in authorization headers for authenticated requests

Technology Stack

- Language: Kotlin
- UI Framework: XML Layouts (AppCompat)
- Networking: Retrofit 2 + OkHttp 3
- Async: Coroutines
- Min SDK: 33 | Target SDK: 34